3.11 图像处理 API 手册

此模块移植自 `openmv`，功能基本一致。详情请参考 官方文档。本文列出与官方 API 的差异部分以及新增 API，同时包含部分原生 API 的参考。

1. 类 `Image`

`Image` 类是机器视觉处理中的基础对象。此类支持从 Micropython GC、MMZ、系统堆、VB 区域等内存区域创建图像对象。此外，还可以通过引用外部内存直接创建图像（ALLOC_REF）。未使用的图像对象会在垃圾回收时自动释放，也可以手动释放内存。

支持的图像格式如下：

- BINARY
- GRAYSCALE
- RGB565
- BAYER
- YUV422
- JPEG
- PNG
- ARGB8888（新增）
- RGB888（新增）
- RGBP888（新增）
- YUV420（新增）

支持的内存分配区域：

- ALLOC_MPGC：Micropython 管理的内存
- ALLOC_HEAP：系统堆内存
- ALLOC_MMZ：多媒体内存
- ALLOC_VB：视频缓冲区
- ALLOC_REF：使用引用对象的内存，不分配新内存

1.1 构造函数

```
image.Image(path, alloc=ALLOC_MMZ, cache=True, phyaddr=0, virtaddr=0, poolid=0,
data=None)
```

从文件路径 `path` 创建图像对象，支持 BMP、PGM、PPM、JPG、JPEG 格式。

```
image.Image(w, h, format, alloc=ALLOC_MMZ, cache=True, phyaddr=0, virtaddr=0,
poolid=0, data=None)
```

创建指定大小和格式的图像对象。

- w：图像宽度
- h：图像高度
- format：图像格式
- alloc：内存分配方式（默认 ALLOC_MMZ）
- cache：是否启用内存缓存（默认启用）
- phyaddr：物理内存地址，仅适用于 VB 区域

- virtaddr：虚拟内存地址，仅适用于 VB 区域
- poolid：VB 区域的池 ID，仅适用于 VB 区域
- data：引用外部数据对象（可选）

示例：

```
# 在 MMZ 区域创建 ARGB8888 格式的 640x480 图像
img = image.Image(640, 480, image.ARGB8888)

# 在 VB 区域创建 YUV420 格式的 640x480 图像
img = image.Image(640, 480, image.YUV420, alloc=image.ALLOC_VB, phyaddr=xxx,
virtaddr=xxx, poolid=xxx)

# 使用外部引用创建 RGB888 格式的 640x480 图像
img = image.Image(640, 480, image.RGB888, alloc=image.ALLOC_REF,
data=buffer_obj)
```

手动释放图像内存：

```
del img
gc.collect()
```

1.2 `phyaddr`

获取图像数据的物理内存地址。

```
image.phyaddr()
```

1.3 `virtaddr`

获取图像数据的虚拟内存地址。

```
image.virtaddr()
```

1.4 `poolid`

获取图像的 VB 池 ID。

```
image.poolid()
```

1.5 `to_rgb888`

将图像转换为 RGB888 格式，返回新图像对象。

```
image.to_rgb888(x_scale=1.0, y_scale=1.0, roi=None, rgb_channel=-1, alpha=256,
color_palette=None, alpha_palette=None, hint=0, alloc=ALLOC_MMZ, cache=True,
phyaddr=0, virtaddr=0, poolid=0)
```

1.6 `copy_from`

将 `src_img` 的内容复制到当前图像对象中。

```
image.copy_from(src_img)
```

1.7 `copy_to`

将当前图像对象内容复制到 `dst_img`。

```
image.copy_to(dst_img)
```

1.8 `to_numpy_ref`

将图像对象转换为 NumPy 数组，返回的 NumPy 数组与原图像对象共享内存。

```
image.to_numpy_ref()
```

支持的格式：GRAYSCALE、RGB565、ARGB8888、RGB888、RGBP888。

1.9 `draw_string_advanced`

增强版 `draw_string`，支持中文显示，并允许用户通过 `font` 参数自定义字体。

```
image.draw_string_advanced(x, y, char_size, str, [color, font])
```

1.10 实现有差异的方法

1.10.1 移除 `crop` 参数的 API

如下这些 API 的 `crop` 参数无效，增加了内存分配方式参数，并总是返回新图像对象。

- `to_bitmap` 方法
- `to_grayscale` 方法
- `to_rgb565` 方法
- `to_rainbow` 方法
- `to_ironbow` 方法
- `to_jpeg` 方法
- `to_png` 方法
- `copy` 方法
- `crop` 方法
- `scale` 方法

1.10.2 画图 API

新增对 `ARGB8888` 和 `RGB888` 格式的支持，其他格式不支持。

1.10.3 BINARY API

`binary` 新增了内存分配方式参数，仅在 `copy=True` 时生效。

1.10.4 POOL API

`mean_pooled` 和 `midpoint_pooled` 方法新增了内存分配方式参数。

1.10.5 **其他图像算法**

这些算法仅支持原生图像格式，`RGB888` 格式需转换后方可使用。

1.11 `width`

```
image.width()
```

返回图像的宽度，以像素为单位。

1.12 `height`

```
image.height()
```

返回图像的高度，以像素为单位。

1.13 `format`

```
image.format()
```

返回图像的格式，可能的值包括：

- `sensor.GRAYSCALE`：灰度图像
- `sensor.RGB565`：RGB图像
- `sensor.JPEG`：JPEG压缩图像

1.14 `size`

```
image.size()
```

返回图像的大小，以字节为单位。

1.15 `get_pixel`

```
image.get_pixel(x, y[, rgbtuple])
```

根据图像的格式获取指定位置 `(x, y)` 处的像素值：

- 对于灰度图像，返回灰度值。
- 对于RGB565图像，返回RGB888格式的元组 `(r, g, b)`。
- 对于Bayer图像，返回该位置的像素值。

不支持压缩图像。

> `get_pixel()` 和 `set_pixel()` 是操作Bayer模式图像的唯一方法。Bayer模式图像是一种特殊的格式，在偶数行包含 `R/G/R/G/...`，奇数行包含 `G/B/G/B/...` 像素，每个像素占用8位。

1.16 `set_pixel`

```
image.set_pixel(x, y, pixel)
```

设置图像指定位置 `(x, y)` 的像素值：

- 对于灰度图像，设置灰度值。
- 对于RGB图像，设置为RGB888格式的元组 `(r, g, b)`。

不支持压缩图像。

> `get_pixel()` 和 `set_pixel()` 是操作Bayer模式图像的唯一方法。

1.17 `mean_pool`

```
image.mean_pool(x_div, y_div)
```

将图像划分为 `x_div * y_div` 的矩形区域，并计算每个区域的平均值，返回包含这些平均值的修改后图像。

该方法可用于快速缩小图像。

不支持压缩图像和Bayer图像。

1.18 `mean_pooled`

```
image.mean_pooled(x_div, y_div)
```

与 `mean_pool()` 类似，但返回的是一个新的图像副本。

1.19 `midpoint_pool`

```
image.midpoint_pool(x_div, y_div[, bias=0.5])
```

在图像中划分 `x_div * y_div` 的区域，并计算每个区域的中点值，返回包含这些中点值的修改后图像。

- 参数 `bias=0.0` 时返回区域的最小值，`bias=1.0` 时返回区域的最大值。

1.20 `midpoint_pooled`

```
image.midpoint_pooled(x_div, y_div[, bias=0.5])
```

类似 `midpoint_pool()`，但返回一个新的图像副本。

1.21 `to_grayscale`

```
image.to_grayscale([copy=False])
```

将图像转换为灰度图像。如果 `copy=True`，会在堆上创建新的图像副本。返回图像对象。

1.22 `to_rgb565`

```
image.to_rgb565([copy=False])
```

将图像转换为RGB565格式彩色图像。如果 `copy=True`，会在堆上创建新的图像副本。返回图像对象。

1.23 `to_rainbow`

```
image.to_rainbow([copy=False])
```

将图像转换为彩虹色图像。返回图像对象。

1.24 `compress`

```
image.compress([quality=50])
```

以指定的质量 `quality`（0-100）对图像进行JPEG压缩。

1.25 `compress_for_ide`

```
image.compress_for_ide([quality=50])
```

对图像进行压缩，并格式化以便在OpenMV IDE中显示。

1.26 `compressed`

```
image.compressed([quality=50])
```

返回JPEG压缩后的图像。

1.27 `compressed_for_ide`

```
image.compressed_for_ide([quality=50])
```

返回JPEG压缩后的图像，格式化以便在OpenMV IDE中显示。

1.28 `copy`

```
image.copy([roi[, copy_to_fb=False]])
```

创建图像对象的副本，支持指定感兴趣区域 `roi`。

1.29 `save`

```
image.save(path[, roi[, quality=50]])
```

将图像保存到指定路径 `path`，支持指定感兴趣区域 `roi` 及JPEG压缩质量 `quality`。

1.30 `clear`

```
image.clear()
```

快速将图像中的所有像素设置为零。返回图像对象。

1.31 `draw_line`

```
image.draw_line(x0, y0, x1, y1[, color[, thickness=1]])
```

在图像上绘制一条从 `(x0, y0)` 到 `(x1, y1)` 的直线。参数可以分别传入 `x0, y0, x1, y1`，也可以将这些值作为元组 `(x0, y0, x1, y1)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- thickness: 控制线条的像素宽度，默认为 1。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.32 `draw_rectangle`

```
image.draw_rectangle(x, y, w, h[, color[, thickness=1[, fill=False]]])
```

在图像上绘制一个矩形。可以分别传入参数 `x, y, w, h`，也可以作为元组 `(x, y, w, h)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- thickness: 控制矩形边框的像素宽度，默认为 1。
- fill: 设置为 `True` 时，将填充矩形内部，默认为 `False`。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.33 `draw_ellipse`

```
image.draw_ellipse(cx, cy, rx, ry, rotation[, color[, thickness=1[,
fill=False]]])
```

在图像上绘制椭圆。参数可以分别传入 `cx, cy, rx, ry, rotation`，也可以作为元组 `(cx, cy, rx, ry, rotation)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。

- thickness: 控制椭圆边框的像素宽度，默认为 1。
- fill: 设置为 `True` 时，将填充椭圆内部，默认为 `False`。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.34 `draw_circle`

```
image.draw_circle(x, y, radius[, color[, thickness=1[, fill=False]]])
```

在图像上绘制一个圆形。参数可以分别传入 `x, y, radius`，也可以作为元组 `(x, y, radius)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- thickness: 控制圆形边框的像素宽度，默认为 1。
- fill: 设置为 `True` 时，将填充圆形内部，默认为 `False`。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.35 `draw_string`

```
image.draw_string(x, y, text[, color[, scale=1[, x_spacing=0[, y_spacing=0[,
mono_space=True]]]]])
```

从图像的 `(x, y)` 位置开始绘制 8x10 大小的文本。参数可以分别传入 `x, y`，也可以作为元组 `(x, y)` 一起传递。

- text: 要绘制的字符串，换行符 `\n`、`\r` 或 `\r\n` 用于将光标移动到下一行。
- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- scale: 控制文本的缩放比例，默认为 1。只能为整数。
- x_spacing: 调整字符之间的水平间距。正值表示增加间距，负值表示减少。
- y_spacing: 调整行之间的垂直间距。正值表示增加间距，负值表示减少。
- mono_space: 默认为 `True`，使字符具有固定宽度。设置为 `False` 时，字符间距将根据字符宽度动态调整。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.36 `draw_cross`

```
image.draw_cross(x, y[, color[, size=5[, thickness=1]]])
```

在图像上绘制一个十字标记。参数可以分别传入 `x, y`，也可以作为元组 `(x, y)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- size: 控制十字标记的大小，默认为 5。
- thickness: 控制十字线条的像素宽度，默认为 1。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.37 `draw_arrow`

```
image.draw_arrow(x0, y0, x1, y1[, color[, thickness=1]])
```

在图像上绘制从 `(x0, y0)` 到 `(x1, y1)` 的箭头。参数可以分别传入 `x0, y0, x1, y1`，也可以作为元组 `(x0, y0, x1, y1)` 一起传递。

- color: 表示颜色的 RGB888 元组，适用于灰度或 RGB565 图像，默认为白色。对于灰度图像，还可以传递像素值（范围 0-255）；对于 RGB565 图像，可以传递字节翻转的 RGB565 值。
- thickness: 控制箭头线条的像素宽度，默认为 1。

该方法返回图像对象，允许通过链式调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.38 `draw_image`

```
image.draw_image(image, x, y[, x_scale=1.0[, y_scale=1.0[, mask=None[,
alpha=256]]]])
```

该函数用于将图像绘制到指定位置 (x, y)，即图像的左上角与该位置对齐。参数 x 和 y 可以分别传递，也可以作为一个元组 (x, y) 传入。

- `x_scale`：控制图像在水平方向的缩放比例（浮点数）。
- `y_scale`：控制图像在垂直方向的缩放比例（浮点数）。
- `mask`：应用于绘制操作的像素级掩码图像。该掩码应为二值图像（黑白像素），且尺寸应与目标图像相同。
- `alpha`：设置源图像绘制时的透明度，取值范围为 0-256。256 表示完全不透明，较小的值表示源图像与目标图像的混合程度，0 则表示完全不修改目标图像。

该方法不支持压缩图像和 Bayer 格式图像。

1.39 `draw_keypoints`

```
image.draw_keypoints(keypoints[, color[, size=10[, thickness=1[, fill=False]]]])
```

在图像上绘制特征点。

- `color`：指定颜色，适用于灰度或 RGB565 图像。默认为白色。对于灰度图像，可以传递灰度值（0-255）；对于 RGB565 图像，可以传递反向字节序的 RGB565 值。
- `size`：控制特征点的大小。
- `thickness`：控制线条的粗细（以像素为单位）。
- `fill`：如果为 True，则填充特征点。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.40 `flood_fill`

```
image.flood_fill(x, y[, seed_threshold=0.05[, floating_threshold=0.05[, color[,
invert=False[, clear_background=False[, mask=None]]]]]])
```

从位置 (x, y) 开始对图像区域进行填充。`x` 和 `y` 可分别传递，也可作为元组 (x, y) 传入。

- `seed_threshold`：控制填充区域内像素与种子像素的差异。
- `floating_threshold`：控制填充区域内相邻像素之间的差异。
- `color`：填充颜色，适用于灰度或 RGB565 图像。默认为白色，也可以传递灰度值或反向字节序的 RGB565 值。
- `invert`：如果设为 True，则反转填充逻辑，即填充除目标区域外的部分。
- `clear_background`：如果设为 True，则将未填充的像素清零。
- `mask`：用于限制填充区域的像素级掩码图像，掩码应为二值图像（黑白像素），尺寸与目标图像相同。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.41 `binary`

```
image.binary(thresholds[, invert=False[, zero=False[, mask=None]]])
```

根据指定的阈值列表 `thresholds`，将图像中的所有像素转换为黑白二值图像。

- `thresholds`：为一个元组列表，格式为 `[(lo, hi), ...]`。对于灰度图像，每个元组定义一个灰度值范围（最低值和最高值）；对于 RGB565 图像，每个元组包含六个值，分别表示 LAB 空间中 L、A 和 B 通道的范围。
- `invert`：如果设为 True，则反转阈值操作，将阈值之外的像素转换为白色。
- `zero`：如果设为 True，则将匹配阈值的像素设置为零，而保留其余像素。
- `mask`：应用于二值化操作的掩码图像。掩码应为二值图像，且尺寸与目标图像相同。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.42 `invert`

```
image.invert()
```

快速反转二值图像中的像素值，即将 0（黑色）变为 1（白色），1（白色）变为 0（黑色）。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.43 `b_and`

```
image.b_and(image[, mask=None])
```

对两个图像进行按位与运算。

- `image`：可以是图像对象、未压缩图像文件的路径（bmp/pgm/ppm），或标量值。对于标量值，可以是 RGB888 元组或灰度图像的基础像素值（例如，8 位灰度值）。
- `mask`：用于限制操作的掩码图像，掩码应为二值图像，且尺寸与目标图像相同。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.44 `b_nand`

```
image.b_nand(image[, mask=None])
```

对两个图像进行按位与非（NAND）运算。

其他参数说明同 `b_and`。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.45 `b_or`

```
image.b_or(image[, mask=None])
```

对两个图像进行按位或运算。

其他参数说明同 `b_and`。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.46 `b_nor`

```
image.b_nor(image[, mask=None])
```

对两个图像进行按位或非（NOR）运算。

其他参数说明同 `b_and`。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.47 `b_xor`

```
image.b_xor(image[, mask=None])
```

对两个图像进行按位异或（XOR）运算。

其他参数说明同 `b_and`。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.48 `b_xnor`

```
image.b_xnor(image[, mask=None])
```

对两个图像进行按位同或（XNOR）运算。

其他参数说明同 `b_and`。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.49 `erode`

```
image.erode(size[, threshold[, mask=None]])
```

通过删除分割区域边缘的像素来实现腐蚀操作。

- `size`：定义腐蚀操作的卷积核大小为 `((size*2)+1)x((size*2)+1)`。
- `threshold`：如果未指定，则执行标准腐蚀操作；如果指定，则根据相邻像素的总和小于阈值来决定腐蚀像素。

返回图像对象，以便后续方法可以链式调用。

1.50 `dilate`

```
image.dilate(size[, threshold[, mask=None]])
```

通过向分割区域边缘添加像素来实现膨胀操作。

其他参数说明同 `erode`。

返回图像对象，以便后续方法可以链式调用。

1.51 `open`

```
image.open(size[, threshold[, mask=None]])
```

按顺序对图像执行腐蚀和膨胀操作。有关详细信息，请参阅 `erode()` 和 `dilate()` 方法。

返回图像对象，以便后续方法可以链式调用。

1.52 `close`

```
image.close(size[, threshold[, mask=None]])
```

按顺序对图像执行膨胀和腐蚀操作。有关详细信息，请参阅 `erode()` 和 `dilate()` 方法。

返回图像对象，以便后续方法可以链式调用。

1.53 `top_hat`

```
image.top_hat(size[, threshold[, mask=None]])
```

该函数返回原图像与执行 `image.open()` 操作后图像的差异。

- `size`：定义操作的卷积核大小为 `((size*2)+1)x((size*2)+1)`。
- `threshold`：用于控制操作强度，若未指定，默认执行标准操作。
- `mask`：像素级掩码图像，掩码应为二值图像（黑白像素），且尺寸应与目标图像一致。仅掩码中为白色的像素会被操作。

该方法不支持压缩图像和 Bayer 格式图像。

1.54 `black_hat`

```
image.black_hat(size[, threshold[, mask=None]])
```

该函数返回原图像与执行 `image.close()` 操作后图像的差异。

- `size`：定义操作的卷积核大小为 `((size*2)+1)x((size*2)+1)`。
- `threshold`：用于控制操作强度，若未指定，默认执行标准操作。
- `mask`：像素级掩码图像，掩码应为二值图像，尺寸需与目标图像一致。仅掩码中为白色的像素会被操作。

该方法不支持压缩图像和 Bayer 格式图像。

1.55 `negate`

```
image.negate()
```

快速反转图像中的所有像素值，即对每个颜色通道的像素进行数值翻转（例如：`255 - pixel`）。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.56 `replace`

```
image.replace(image[, hmirror=False[, vflip=False[, mask=None]]])
```

该函数将指定图像替换到当前图像上。

- `image`：可以是图像对象、未压缩的图像文件路径（bmp/pgm/ppm），或标量值。标量值可以是 RGB888 元组或基础像素值（例如，灰度图像的 8 位灰度值）。
- `hmirror`：为 True 时，水平镜像替换图像。
- `vflip`：为 True 时，垂直翻转替换图像。
- `mask`：像素级掩码图像，掩码应为二值图像，尺寸需与目标图像一致。仅掩码中为白色的像素会被修改。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.57 `add`

```
image.add(image[, mask=None])
```

将两个图像按像素进行加法运算。

- `image`：可以是图像对象、未压缩的图像文件路径（bmp/pgm/ppm），或标量值。标量值可以是 RGB888 元组或基础像素值（例如，灰度图像的 8 位灰度值）。
- `mask`：像素级掩码图像，掩码应为二值图像，且尺寸应与目标图像一致。仅掩码中为白色的像素会被修改。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.58 `sub`

```
image.sub(image[, reverse=False[, mask=None]])
```

将两个图像按像素进行减法运算。

- `image`：可以是图像对象、未压缩的图像文件路径（bmp/pgm/ppm），或标量值。标量值可以是 RGB888 元组或基础像素值（例如，灰度图像的 8 位灰度值）。
- `reverse`：为 True 时，反转减法运算顺序，即由 `this_image - image` 改为 `image - this_image`。
- `mask`：像素级掩码图像，掩码应为二值图像，且尺寸应与目标图像一致。仅掩码中为白色的像素会被修改。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.59 `mul`

```
image.mul(image[, invert=False[, mask=None]])
```

将两个图像按像素进行乘法运算。

- `image`：可以是图像对象、未压缩的图像文件路径（bmp/pgm/ppm），或标量值。标量值可以是 RGB888 元组或基础像素值（例如，灰度图像的 8 位灰度值）。
- `invert`：为 True 时，乘法运算将由 `a * b` 变为 `1 / ((1 / a) * (1 / b))`，这会使图像变亮而非变暗（类似于"屏幕混合"效果）。
- `mask`：像素级掩码图像，掩码应为二值图像，且尺寸应与目标图像一致。仅掩码中为白色的像素会被修改。

返回图像对象，以便后续方法可以链式调用。

该方法不支持压缩图像和 Bayer 格式图像。

1.60 `div`

```
image.div(image[, invert=False[, mask=None]])
```

将当前图像与另一个图像进行像素级除法操作。

`image` 参数可以是图像对象、未压缩的图像文件路径（支持 bmp/pgm/ppm 格式），或标量值。若为标量值，支持 RGB888 元组或基本像素值（例如，8 位灰度级图像的灰度值，或 RGB 图像的反转 RGB565 值）。

设置 `invert=True` 可以将除法的顺序由 `a/b` 改为 `b/a`。

`mask` 是一个用于像素级操作的掩码图像。掩码图像应为黑白图像，且尺寸必须与当前图像相同。仅修改掩码中设置为白色的像素。

返回修改后的图像对象，允许通过点操作符调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.61 `min`

```
image.min(image[, mask=None])
```

用两个图像在像素级的最小值替换当前图像中的像素。

`image` 参数可以是图像对象、未压缩的图像文件路径（支持 bmp/pgm/ppm 格式），或标量值。若为标量值，支持 RGB888 元组或基本像素值（如 8 位灰度图像的灰度值，或 RGB 图像的反转 RGB565 值）。

`mask` 是用于绘图操作的像素级掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回新的图像对象，以便后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

此方法在 OpenMV4 上不可用。

1.62 `max`

```
image.max(image[, mask=None])
```

用两个图像在像素级的最大值替换当前图像中的像素。

`image` 参数可以是图像对象、未压缩的图像文件路径（支持 bmp/pgm/ppm 格式），或标量值。若为标量值，支持 RGB888 元组或基本像素值（如 8 位灰度图像的灰度值，或 RGB 图像的反转 RGB565 值）。

`mask` 是用于绘图操作的像素级掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回新的图像对象，以便后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.63 `difference`

```
image.difference(image[, mask=None])
```

对两个图像的像素值取绝对差。每个颜色通道的像素值按如下方式更新：`ABS(this.pixel - image.pixel)`。

`image` 参数可以是图像对象、未压缩的图像文件路径（支持 bmp/pgm/ppm 格式），或标量值。若为标量值，支持 RGB888 元组或基本像素值（如 8 位灰度图像的灰度值，或 RGB 图像的反转 RGB565 值）。

`mask` 是用于绘图操作的像素级掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回修改后的图像对象，允许后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.64 `blend`

```
image.blend(image[, alpha=128[, mask=None]])
```

将另一张图像与当前图像进行融合操作。

`image` 参数可以是图像对象、未压缩的图像文件路径（支持 bmp/pgm/ppm 格式），或标量值。若为标量值，支持 RGB888 元组或基本像素值（如 8 位灰度图像的灰度值，或 RGB 图像的反转 RGB565 值）。

`alpha` 控制混合比例，取值范围为 0 到 256。值越接近 0，融合程度越高；接近 256 则相反。

`mask` 是用于像素级操作的掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回融合后的图像对象，允许后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.65 `histeq`

```
image.histeq([adaptive=False[, clip_limit=-1[, mask=None]]])
```

对图像执行直方图均衡化，使图像的对比度和亮度标准化。

若 `adaptive=True`，则启用自适应直方图均衡化方法，通常比非自适应方法效果更好，但处理速度较慢。

`clip_limit` 用于控制自适应直方图均衡化的对比度，较小的值（如 10）可以生成对比度受限的图像。

`mask` 是用于像素级操作的掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回处理后的图像对象，允许后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.66 `mean`

```
image.mean(size[, threshold=False[, offset=0[, invert=False[, mask=None]]]])
```

使用盒式滤波器对图像进行标准均值模糊处理。

`size` 指定滤波器的大小，取值为 1（对应 3x3 内核）、2（对应 5x5 内核）或更大。

若希望对滤波器输出进行自适应阈值处理，可设置 `threshold=True`，根据相邻像素的亮度对目标像素进行二值化处理。负值的 `offset` 参数会导致更多像素设置为 1，正值则只会设置对比度最强的像素为 1。设置 `invert=True` 以反转输出的二值图像。

`mask` 是用于像素级操作的掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回模糊处理后的图像对象，允许后续调用其他方法。

不支持压缩图像和 Bayer 格式图像。

1.67 `median`

```
image.median(size, percentile=0.5[, threshold=False[, offset=0[, invert=False[,
mask=None]]]])
```

对图像应用中值滤波，该滤波器能够在保持边缘细节的前提下平滑图像，但处理速度较慢。

`size` 指定滤波器的大小，取值为 1（对应 3x3 内核）、2（对应 5x5 内核）或更大。

`percentile` 参数控制所选像素的百分位数。默认情况下，选择中间值（第 50 百分位数）。将 `percentile` 设置为 0 可实现最小滤波，设置为 1 则实现最大滤波。

如果希望对滤波结果进行自适应阈值处理，可传入 `threshold=True`。参数 `offset` 和 `invert` 控制二值化处理的输出效果。

`mask` 是用于像素级操作的掩码图像，必须是黑白图像，且大小需与当前图像相同。仅掩码中白色像素对应的区域会被修改。

返回滤波后的图像对象。

不支持压缩图像和 Bayer 格式图像。

1.68 `mode`

```
image.mode(size[, threshold=False, offset=0, invert=False, mask])
```

在图像上应用众数滤波，通过相邻像素的模式替换每个像素。该方法在灰度图像中效果良好，但由于其非线性特性，可能在RGB图像的边缘处产生伪影。

参数说明：

- size：内核的大小，取值为1（3x3内核）或2（5x5内核）。
- threshold：若设置为`True`，则启用自适应阈值处理，依据环境像素的亮度调整像素值（设置为1或0）。
- offset：负值可增加被设置为1的像素数量，正值则限制为仅设置最强对比度的像素为1。
- invert：设置为`True`以反转输出的二进制图像结果。

- mask：用于绘图操作的像素级掩码图像，要求掩码图像仅包含黑色或白色像素，并且尺寸需与待处理的图像相同。只有掩码中设置的像素会被修改。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.69 `midpoint`

```
image.midpoint(size[, bias=0.5, threshold=False, offset=0, invert=False, mask])
```

在图像上应用中点滤波，针对每个像素邻域计算中点值((max-min)/2)。

参数说明：

- size：内核的大小，取值为1（3x3内核）、2（5x5内核）或更高。
- bias：控制图像混合的最小/最大程度，0仅进行最小滤波，1仅进行最大滤波，值介于0和1之间则可进行混合滤波。
- threshold：若设置为`True`，启用自适应阈值处理，依据环境像素的亮度调整像素值（设置为1或0）。
- offset：负值可增加被设置为1的像素数量，正值则限制为仅设置最强对比度的像素为1。
- invert：设置为`True`以反转输出的二进制图像结果。
- mask：用于绘图操作的像素级掩码图像，要求掩码图像仅包含黑色或白色像素，并且尺寸需与待处理的图像相同。只有掩码中设置的像素会被修改。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.70 `morph`

```
image.morph(size, kernel, mul=Auto, add=0)
```

通过指定的卷积内核对图像进行卷积操作，实现通用卷积。

参数说明：

- size：内核的大小，控制为((size*2)+1)x((size*2)+1)像素。
- kernel：用于卷积的内核，可以是一个元组或一个取值范围为[-128:127]的列表。
- mul：用于与卷积结果相乘的数字，若不设置，则使用默认值以防止卷积输出缩放。
- add：用于与每个像素的卷积结果相加的数值。

`mul`可用于全局对比度调整，`add`可用于全局亮度调整。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.71 `gaussian`

```
image.gaussian(size[, unsharp=False[, mul[, add=0[, threshold=False[, offset=0[,
invert=False[, mask=None]]]]]]])
```

通过高斯核平滑图像进行卷积。

参数说明：

- size：内核的大小，取值为1（3x3内核）、2（5x5内核）或更高。
- unsharp：若设置为`True`，则执行非锐化掩模操作，增强图像边缘的清晰度。
- mul：用于与卷积结果相乘的数字，若不设置，则使用默认值以防止卷积输出缩放。
- add：用于与每个像素的卷积结果相加的数值。

`mul`可用于全局对比度调整，`add`可用于全局亮度调整。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.72 `laplacian`

```
image.laplacian(size[, sharpen=False[, mul[, add=0[, threshold=False[,
offset=0[, invert=False[, mask=None]]]]]]])
```

通过拉普拉斯核进行边缘检测，对图像进行卷积。

参数说明：

- size：内核的大小，取值为1（3x3内核）、2（5x5内核）或更高。
- sharpen：若设置为`True`，则对图像进行锐化，而非仅输出未经过阈值处理的边缘图像。
- mul：用于与卷积结果相乘的数字，若不设置，则使用默认值以防止卷积输出缩放。
- add：用于与每个像素的卷积结果相加的数值。

`mul`可用于全局对比度调整，`add`可用于全局亮度调整。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.73 `bilateral`

```
image.bilateral(size[, color_sigma=0.1[, space_sigma=1[, threshold=False[,
offset=0[, invert=False[, mask=None]]]]]])
```

通过双边滤波器对图像进行卷积，平滑图像的同时保持边缘特征。

参数说明：

- size：内核的大小，取值为1（3x3内核）、2（5x5内核）或更高。
- color_sigma：控制双边滤波器匹配颜色的接近程度，增加该值将导致颜色模糊增加。
- space_sigma：控制像素在空间方面模糊的程度，增加该值将导致像素模糊增强。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.74 `cartoon`

```
image.cartoon(size[, seed_threshold=0.05[, floating_threshold=0.05[,
mask=None]]])
```

通过使用Flood-Fill算法对图像进行处理，使图像中的所有像素区域被填充，从而有效去除纹理。

参数说明：

- size：控制填充区域的大小。

- seed_threshold：控制填充区域内像素与原始起始像素之间的差异。
- floating_threshold：控制填充区域内像素与相邻像素之间的差异。

返回值：返回图像对象，以便进一步调用其他方法。

注意：不支持压缩图像和Bayer图像。

1.75 `remove_shadows`

```
image.remove_shadows([image])
```

从当前图像中移除阴影。

功能说明：

- 若当前图像没有“无阴影”版本，该方法将尝试从图像中去除阴影，适用于平坦均匀背景中的阴影去除。
- 如果当前图像有“无阴影”版本，则此方法将依据“真实源”背景图像去除阴影，同时保留非阴影像素，以便于新对象的添加。

返回值：返回图像对象，以便进一步调用其他方法。

注意：仅支持RGB565图像。

1.76 `chrominvar`

```
image.chrominvar()
```

去除图像中的照明效果，仅保留颜色渐变。此方法速度较快，但对阴影存在一定的敏感性。

返回值：返回图像对象，以便进一步调用其他方法。

注意：仅支持RGB565图像。

1.77 `illuminvar`

```
image.illuminvar()
```

从图像中去除照明效果，仅保留颜色渐变。该方法速度较慢，但对阴影不敏感。

返回值：返回图像对象，以便进一步调用其他方法。

注意：仅支持RGB565图像。

1.78 `linpolar`

```
image.linpolar([reverse=False])
```

此方法用于将图像从笛卡尔坐标系重新投影至线性极坐标系。

- **参数说明**：
  - `reverse`：布尔值，默认为 `False`。若设置为 `True`，则会在相反方向进行重新投影。

线性极坐标的重新投影过程将图像的旋转转换为 x 方向的平移。

注意：此功能不支持压缩图像。

1.79 `logpolar`

```
image.logpolar([reverse=False])
```

该方法用于将图像从笛卡尔坐标系重新投影至对数极坐标系。

- **参数说明**：
  - `reverse`：布尔值，默认为 `False`。若设置为 `True`，则会在相反方向进行重新投影。

对数极坐标的重新投影过程将图像的旋转转换为 x 方向的平移和 y 方向的缩放。

注意：此功能不支持压缩图像。

1.80 `lens_corr`

```
image.lens_corr([strength=1.8[, zoom=1.0]])
```

该方法用于进行镜头畸变校正，以消除镜头导致的鱼眼效果。

- **参数说明**：
  - `strength`：浮点数，决定去除鱼眼效果的程度。默认值为 1.8，用户可以根据图像效果进行调整。
  - `zoom`：浮点数，用于图像缩放，默认值为 1.0。

该方法返回图像对象，以便用户可以继续调用其他方法。

注意：此功能不支持压缩图像和 Bayer 图像。

1.81 `rotation_corr`

```
img.rotation_corr([x_rotation=0.0[, y_rotation=0.0[, z_rotation=0.0[,
x_translation=0.0[, y_translation=0.0[, zoom=1.0[, fov=60.0[, corners]]]]]]])
```

通过对帧缓冲区进行 3D 旋转，来校正图像中的透视问题。

- **参数说明**：
  - `x_rotation`：图像绕 x 轴旋转的角度（上下旋转）。
  - `y_rotation`：图像绕 y 轴旋转的角度（左右旋转）。
  - `z_rotation`：图像绕 z 轴旋转的角度（图像朝向的调整）。
  - `x_translation`：图像旋转后沿 x 轴移动的单位，单位为 3D 空间的单位。
  - `y_translation`：图像旋转后沿 y 轴移动的单位，单位为 3D 空间的单位。
  - `zoom`：图像缩放倍数，默认为 1.0。
  - `fov`：用于 2D->3D 投影的视场。当此值接近 0 时，图像位于视口无限远处；当此值接近 180 时，图像位于视口中。通常情况下不建议更改此值，但可以通过调整它来改变 2D->3D 映射效果。
  - `corners`：包含四个 `(x, y)` 元组的列表，表示四个角点，用于创建四点对应单应性，将第一个角点映射到 (0,0)，第二个角点映射到 (image_width-1, 0)，第三个角点映射到 (image_width-1, image_height-1)，第四个角点映射到 (0, image_height-1)。此参数允许用户使用 `rotation_corr` 实现鸟瞰图转换。

该方法返回图像对象，以便用户可以继续调用其他方法。

注意：此功能不支持压缩图像或 Bayer 图像。

1.82 `get_similarity`

```
image.get_similarity(image)
```

此方法返回一个“相似度”对象，利用 SSIM 算法描述两幅图像之间的相似度，基于 8x8 像素色块进行比较。

- **参数说明**：
  - `image`：可以是图像对象、未压缩图像文件路径（bmp/pgm/ppm），或标量值。如果为标量值，则可以是 RGB888 元组或基础像素值（如灰度图像的 8 位灰度级或 RGB 图像的字节反转 RGB565 值）。

注意：此功能不支持压缩图像和 Bayer 图像。

1.83 `get_histogram`

```
image.get_histogram([thresholds[, invert=False[, roi[, bins[, l_bins[, a_bins[, b_bins]]]]]]])
```

此方法在 ROI 的所有颜色通道上进行标准化直方图运算，并返回 histogram 对象。有关 histogram 对象的详细信息，请参阅相应文档。用户也可以使用 `image.get_hist` 或 `image.histogram` 调用此方法。

- **参数说明**：
  - `thresholds`：元组列表，定义要追踪的颜色范围。对于灰度图像，每个元组需要包含两个值（最小和最大灰度值）；对于 RGB565 图像，每个元组需要包含六个值（l_lo，l_hi，a_lo，a_hi，b_lo，b_hi）。
  - `invert`：布尔值，默认为 `False`。若设置为 `True`，则反转阈值操作，像素将在已知颜色范围之外进行匹配。
  - `roi`：感兴趣区域的矩形元组 `(x, y, w, h)`，如果未指定，则为整个图像。
  - `bins`：用于灰度图像的箱数，或用于 RGB565 图像的各个通道的箱数。

注意：此功能不支持压缩图像和 Bayer 图像。

1.84 `get_statistics`

```
image.get_statistics([thresholds[, invert=False[, roi[, bins[, l_bins[, a_bins[, b_bins]]]]]]])
```

此方法计算 ROI 内每个颜色通道的平均值、中位数、众数、标准偏差、最小值、最大值、下四分位和上四分位，并返回一个数据对象。

- **参数说明**：
  - `thresholds`：元组列表，定义要追踪的颜色范围。对于灰度图像，每个元组需要包含两个值（最小和最大灰度值）；对于 RGB565 图像，每个元组需要包含六个值（l_lo，l_hi，a_lo，a_hi，b_lo，b_hi）。
  - `invert`：布尔值，默认为 `False`。若设置为 `True`，则反转阈值操作，像素将在已知颜色范围之外进行匹配。
  - `roi`：感兴趣区域的矩形元组 `(x, y, w, h)`，如果未指定，则为整个图像。
  - `bins`：用于灰度图像的箱数，或用于 RGB565 图像的各个通道的箱数。

注意：此功能不支持压缩图像和 Bayer 图像。

1.85 `get_regression`

```
image.get_regression(thresholds[, invert=False[, roi[, x_stride=2[, y_stride=1[,
area_threshold=10[, pixels_threshold=10[, robust=False]]]]]]])
```

该方法对图像中所有符合阈值的像素进行线性回归计算。该计算采用最小二乘法，速度较快，但无法处理异常值。若 `robust` 设置为 `True`，则将使用泰尔指数计算像素间斜率的中位数。

- **参数说明**：
  - `thresholds`：元组列表，定义要追踪的颜色范围。
  - `invert`：布尔值，默认为 `False`。若设置为 `True`，则反转阈值操作。
  - `roi`：感兴趣区域的矩形元组 `(x, y, w, h)`，如果未指定，则为整个图像。
  - `x_stride`：调用函数时跳过的 x 像素数。
  - `y_stride`：调用函数时跳过的 y 像素数。
  - `area_threshold`：若回归后的边界框区域小于该值，则返回 `None`。
  - `pixels_threshold`：若回归后的像素数小于该值，则返回 `None`。

该方法返回一个 `image.line` 对象，详细使用可参见以下博文：Linear Regression Line Following。

注意：此功能不支持压缩图像和 Bayer 图像。

1.86 `find_blobs`

```
image.find_blobs(thresholds[, invert=False[, roi[, x_stride=2[, y_stride=1[,
area_threshold=10[, pixels_threshold=10[, merge=False[, margin=0[,
threshold_cb=None[, merge_cb=None]]]]]]]]])
```

此函数用于查找图像中的所有色块，并返回一个包含每个色块对象的列表。有关 `image.blob` 对象的更多信息，请参阅相关文档。

参数 `thresholds` 必须为元组列表，形式为 `[(lo, hi), (lo, hi), ...]`，用于定义需要追踪的颜色范围。对于灰度图像，每个元组应包含两个值：最小灰度值和最大灰度值。函数将仅考虑落在这些阈值之间的像素区域。对于 RGB565 图像，每个元组需要包含六个值 `(l_lo, l_hi, a_lo, a_hi, b_lo, b_hi)`，分别对应 LAB 色彩空间中的 L、A 和 B 通道的最小和最大值。该函数会自动纠正最小值和最大值的交换情况。如果元组包含超过六个值，则其余值将被忽略；若元组不足，则假定缺失的阈值为最大范围。

注释：

要获取目标对象的阈值，只需在 IDE 帧缓冲区内选择（单击并拖动）要跟踪的对象，直方图将实时更新。然后，记录下每个直方图通道中颜色分布的起始与结束位置，这些值将作为 `thresholds` 的低值和高值。建议手动确定阈值，以避免上下四分位数的微小差异。

您还可以通过进入 OpenMV IDE 的“工具” -> “机器视觉” -> “阈值编辑器”，并通过 GUI 界面拖动滑块来确定颜色阈值。

- `invert` 参数用于反转阈值操作，使得仅在已知颜色范围之外的像素被匹配。
- `roi` 参数为感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 将默认为整个图像的矩形。操作仅限于该区域内的像素。
- `x_stride` 为查找色块时需要跳过的 x 像素数量。在找到色块后，直线填充算法将精确处理该区域。如果已知色块较大，可以增加 `x_stride` 以提高查找速度。
- `y_stride` 为查找色块时需要跳过的 y 像素数量。在找到色块后，直线填充算法将精确处理该区域。如果已知色块较大，可以增加 `y_stride` 以提高查找速度。
- `area_threshold` 用于过滤掉边界框区域小于此值的色块。

- `pixels_threshold` 用于过滤掉像素数量少于此值的色块。
- `merge` 若为 `True`，则合并所有未被过滤的色块，这些色块的边界矩形互相重叠。`margin` 可用于在合并测试中增大或减小色块边界矩形的大小。例如，边缘为 1 的两个重叠色块将被合并。

合并色块可实现颜色代码的追踪。每个色块对象具有一个 `code` 值，该值为一个位向量。例如，若在 `image.find_blobs` 中输入两个颜色阈值，则第一个阈值对应的代码为 1，第二个为 2（第三个代码为 4，第四个代码为 8，以此类推）。合并色块时，所有的 `code` 通过逻辑或运算进行合并，以指示产生它们的颜色。这使得同时追踪两种颜色成为可能，若两个颜色得到同一个色块对象，则可能对应于某一种颜色代码。

在使用严格的颜色范围时，可能无法完全追踪目标对象的所有像素，此时可考虑合并色块。若希望合并色块但不希望不同阈值的色块被合并，可以分别调用两次 `image.find_blobs`。

- `threshold_cb` 可设置为在每个色块经过阈值筛选后调用的回调函数，以从即将合并的色块列表中过滤出特定色块。回调函数将接收一个参数：待筛选的色块对象。若希望保留色块，回调函数应返回 `True`，否则返回 `False`。
- `merge_cb` 可设置为在两个即将合并的色块间调用的回调函数，以控制合并的批准或禁止。回调函数将接收两个参数，即两个待合并的色块对象。若希望合并色块，应返回 `True`，否则返回 `False`。

注意： 此功能不支持压缩图像和 Bayer 图像。

1.87 `find_lines`

```
image.find_lines([roi[, x_stride=2[, y_stride=1[, threshold=1000[,
theta_margin=25[, rho_margin=25]]]]]])
```

此函数使用霍夫变换查找图像中的所有直线，并返回一个 `image.line` 对象的列表。

- `roi` 为感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像的矩形。操作仅限于该区域内的像素。
- `x_stride` 为霍夫变换过程中需要跳过的 x 像素数量。如果已知直线较长，可增加 `x_stride`。
- `y_stride` 为霍夫变换过程中需要跳过的 y 像素数量。如果已知直线较长，可增加 `y_stride`。
- `threshold` 控制从霍夫变换中检测到的直线。仅返回大于或等于该阈值的直线。合适的阈值取决于图像内容。请注意，直线的大小（magnitude）是构成直线的所有索贝尔滤波像素大小的总和。
- `theta_margin` 控制检测到的直线的合并，若直线的角度在 `theta_margin` 范围内则进行合并。
- `rho_margin` 同样控制检测到的直线的合并，若直线的 rho 值在 `rho_margin` 范围内则进行合并。

该方法通过在图像上应用索贝尔滤波器，并利用其幅值和梯度响应执行霍夫变换。无需对图像进行任何预处理，尽管图像的清理和过滤将会产生更为稳定的结果。

注意： 此功能不支持压缩图像和 Bayer 图像。

1.88 `find_line_segments`

```
image.find_line_segments([roi[, merge_distance=0[, max_theta_difference=15]]])
```

该函数使用霍夫变换查找图像中的线段，并返回一个 `image.line` 对象的列表。

- `roi` 为感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像的矩形。操作仅限于该区域内的像素。
- `merge_distance` 指定两条线段之间的最大像素距离，若小于该值则合并为一条线段。
- `max_theta_difference` 为需合并的两条线段之间的最大角度差。

该方法使用 LSD 库（OpenCV 亦采用）来查找图像中的线段。虽然速度较慢，但准确性高，且线段不会出现跳跃现象。

注意： 此功能不支持压缩图像和 Bayer 图像。

1.89 `find_circles`

```
image.find_circles([roi[, x_stride=2[, y_stride=1[, threshold=2000[,
x_margin=10[, y_margin=10[, r_margin=10]]]]]]])
```

该函数使用霍夫变换在图像中查找圆形，并返回一个 `image.circle` 对象的列表。

- `roi` 为感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像的矩形。操作仅限于该区域内的像素。
- `x_stride` 为霍夫变换过程中需要跳过的 x 像素数量。如果已知圆较大，可增加 `x_stride`。
- `y_stride` 为霍夫变换过程中需要跳过的 y 像素数量。如果已知圆较大，可增加 `y_stride`。
- `threshold` 控制检测到的圆的大小，仅返回大于或等于该阈值的圆。合适的阈值取决于图像内容。请注意，圆的大小（magnitude）是构成圆的所有索贝尔滤波像素

大小的总和。

- `x_margin` 为对 x 坐标进行合并时允许的最大像素偏差。
- `y_margin` 为对 y 坐标进行合并时允许的最大像素偏差。
- `r_margin` 为对半径进行合并时允许的最大像素偏差。

该方法通过在图像上应用索贝尔滤波器，并利用其幅值和梯度响应执行霍夫变换。无需对图像进行任何预处理，尽管图像的清理和过滤将会产生更为稳定的结果。

注意： 此功能不支持压缩图像和 Bayer 图像。

1.90 `find_rects`

```
image.find_rects([roi=Auto, threshold=10000])
```

此函数使用与 AprilTag 相同的四边形检测算法查找图像中的矩形。该算法最适用于与背景形成鲜明对比的矩形。AprilTag 的四边形检测能够处理任意缩放、旋转和剪切的矩形，并返回一个包含 `image.rect` 对象的列表。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

在返回的矩形列表中，边界大小（通过在矩形边缘的所有像素上滑动索贝尔算子并累加其值）小于 `threshold` 的矩形将被过滤。适当的 `threshold` 值取决于具体的应用场景。

注意： 不支持压缩图像和 Bayer 图像。

1.91 `find_qrcodes`

```
image.find_qrcodes([roi])
```

该函数查找指定 ROI 内的所有二维码，并返回一个包含 `image.qrcode` 对象的列表。有关更多信息，请参考 `image.qrcode` 对象的相关文档。

为了确保该方法成功运行，图像上的二维码需尽量平展。可以通过使用 `sensor.set_windowing` 函数在镜头中心放大、使用 `image.lens_corr` 函数消除镜头的桶形畸变，或更换视野较窄的镜头，获得不受镜头畸变影响的平展二维码。部分机器视觉镜头不产生桶形失真，但其成本高于 OpenMV 提供的标准镜头，这些镜头为无畸变镜头。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

注意： 不支持压缩图像和 Bayer 图像。

1.92 `find_apriltags`

```
image.find_apriltags([roi[, families=image.TAG36H11[, fx[, fy[, cx[, cy]]]]]])
```

该函数查找指定 ROI 内的所有 AprilTag，并返回一个包含 `image.apriltag` 对象的列表。有关更多信息，请参考 `image.apriltag` 对象的相关文档。

与二维码相比，AprilTags 可以在更远的距离、较差的光照条件和更扭曲的图像环境下被有效检测。AprilTags 能够应对各种图像失真问题，而二维码则不能。因此，AprilTags 仅将数字 ID 编码作为其有效载荷。

此外，AprilTags 还可用于定位。每个 `image.apriltag` 对象将返回其三维位置信息和旋转角度。位置信息由 `fx`、`fy`、`cx` 和 `cy` 决定，分别表示图像在 X 和 Y 方向上的焦距和中心点。

> 可以使用 OpenMV IDE 内置的标签生成器工具创建 AprilTags。该工具可生成可打印的 8.5"x11" 格式的 AprilTags。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。
- `families` 是要解码的标签家族的位掩码，以逻辑或形式表示：
  - `image.TAG16H5`
  - `image.TAG25H7`
  - `image.TAG25H9`
  - `image.TAG36H10`
  - `image.TAG36H11`
  - `image.ARTOOLKIT`

默认设置为最常用的 `image.TAG36H11` 标签家族。请注意，启用每个标签家族都会稍微降低 `find_apriltags` 的速度。

- `fx` 是以像素为单位的相机 X 方向的焦距。标准 OpenMV Cam 的值为 ((2.8 / 3.984) \times 656)，该值通过毫米计的焦距值除以 X 方向上感光元件的长度，再乘以 X 方向上感光元件的像素数量（针对 OV7725 感光元件）。
- `fy` 是以像素为单位的相机 Y 方向的焦距。标准 OpenMV Cam 的值为 ((2.8 / 2.952) \times 488)，该值通过毫米计的焦距值除以 Y 方向上感光元件的长度，再乘以 Y 方向上感光元件的像素数量（针对 OV7725 感光元件）。
- `cx` 是图像的中心，即 `image.width()/2`，而非 `roi.w()/2`。
- `cy` 是图像的中心，即 `image.height()/2`，而非 `roi.h()/2`。

注意： 不支持压缩图像和 Bayer 图像。

1.93 `find_datamatrices`

```
image.find_datamatrices([roi[, effort=200]])
```

该函数查找指定 ROI 内的所有数据矩阵，并返回一个包含 `image.datamatrix` 对象的列表。有关更多信息，请参考 `image.datamatrix` 对象的相关文档。

为了确保该方法成功运行，图像上的矩形码需尽量平展。可以通过使用 `sensor.set_windowing` 函数在镜头中心放大、使用 `image.lens_corr` 函数消除镜头的桶形畸变，或更换视野较窄的镜头，获得不受镜头畸变影响的平展矩形码。部分机器视觉镜头不产生桶形失真，但其成本高于 OpenMV 提供的标准镜头，这些镜头为无畸变镜头。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。
- `effort` 控制查找矩形码匹配所需的计算时间。默认值为 200，适用于所有用例。然而，您可以在降低检测率的情况下提高帧速率，或在降低帧速率的情况下提高检测率。请注意，当 `effort` 设置低于约 160 时，无法进行任何检测；相反，您可以将其设置为任何更高的值，但若设置高于 240，检测率将不会进一步提高。

注意： 不支持压缩图像和 Bayer 图像。

1.94 `find_barcodes`

```
image.find_barcodes([roi])
```

该函数查找指定 ROI 内的所有一维条形码，并返回一个包含 `image.barcode` 对象的列表。有关更多信息，请参考 `image.barcode` 对象的相关文档。

为了获得最佳效果，建议使用长为 640 像素、宽为 40/80/160 像素的窗口。窗口的垂直程度越低，运行速度越快。由于条形码是线性一维图像，因此在一个方向上需具有较高分辨率，而在另一个方向上可具有较低分辨率。请注意，该函数会进行水平和垂直扫描，因此您可以使用宽为 40/80/160 像素、长为 480 像素的窗口。请务必调整镜头，使条形码位于焦距最清晰的区域。模糊的条形码无法解码。

该函数支持以下所有一维条形码：

- `image.EAN2`
- `image.EAN5`
- `image.EAN8`
- `image.UPCE`
- `image.ISBN10`
- `image.UPCA`
- `image.EAN13`
- `image.ISBN13`
- `image.I25`
- `image.DATABAR (RSS-14)`
- `image.DATABAR_EXP (RSS-Expanded)`
- `image.CODABAR`
- `image.CODE39`
- `image.PDF417`
- `image.CODE93`
- `image.CODE128`

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

注意： 不支持压缩图像和 Bayer 图像。

1.95 `find_displacement`

```
image.find_displacement(template[, roi[, template_roi[, logpolar=False]]])
```

该函数根据模板查找图像的变换偏移量。此方法可用于实现光流分析。返回结果为一个 `image.displacement`

对象，包含基于相位相关的位移计算结果。

- `roi` 是待处理的矩形区域 `(x, y, w, h)`。若未指定，则默认为整个图像。
- `template_roi` 是待处理的模板区域 `(x, y, w, h)`。若未指定，则默认为整个图像。

`roi` 和 `template_roi` 必须具有相同的宽度和高度，但 `x` 和 `y` 坐标可以位于图像的任意位置。您可以在较大图像上滑动较小的 ROI 以获得光流渐变图像。

`image.find_displacement` 通常用于计算两个图像之间的 X/Y 平移。然而，如果设置 `logpolar=True`，则将同时找到旋转和缩放比例的变化。相同的 `image.displacement` 对象可以提供这两种结果。

注意： 不支持压缩图像和 Bayer 图像。

注解： 请在长宽一致的图像（例如 `sensor.B64X64`）上使用此方法。

1.96 `find_number`

```
image.find_number(roi)
```

该函数利用在 MINST 数据集上训练的 LENET-6 卷积神经网络（CNN）检测图像中任何位置的 28x28 ROI 中的数字。返回一个包含检测到的数字（0-9）和相应置信度（0-1）的元组。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持灰度图像，且为实验性功能。若未来运行基于 Caffe 在 PC 上训练的任何 CNN，该方法可能会被删除。最新版本（3.0.0）固件已移除此函数。

1.97 `classify_object`

```
image.classify_object(roi)
```

此函数利用 CIFAR-10 卷积神经网络 (CNN) 对图像中的感兴趣区域 (ROI) 进行对象分类，能够识别出飞机、汽车、鸟类、猫、鹿、狗、青蛙、马、船和卡车等类别。该方法内部会自动将输入图像缩放至 32x32 像素以供 CNN 处理。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持 RGB565 格式的图像。

注解： 该方法为实验性功能，未来若实施基于 Caffe 在 PC 上训练的 CNN，可能会移除此方法。

1.98 `find_template`

```
image.find_template(template, threshold[, roi[, step=2[,
search=image.SEARCH_EX]]])
```

此函数通过归一化互相关 (NCC) 算法在图像中寻找第一个与模板匹配的位置，并返回匹配位置的边界框元组 `(x, y, w, h)`；若未找到匹配，返回 `None`。

- `template` 是一个小图像对象，需与目标图像对象相匹配。请注意，两个图像均应为灰度图像。
- `threshold` 是一个浮点数（范围为 0.0 至 1.0），较小的值可提高检测速率，但可能增加误报率；相反，较高的值会降低检测速率，同时降低误报率。
- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。
- `step` 指在查找模板时需要跳过的像素数量，跳过像素可显著提高算法的运行速度。该参数适用于 `SEARCH_EX` 模式下的算法。
- `search` 可取 `image.SEARCH_DS` 或 `image.SEARCH_EX`。`image.SEARCH_DS` 搜索模板所用的算法较 `image.SEARCH_EX` 更快，但在模板位于图像边缘时，可能无法成功搜索。`image.SEARCH_EX` 可对图像进行更为详尽的搜索，但其运行速度低于 `image.SEARCH_DS`。

注意： 此方法仅支持灰度图像。

1.99 `find_features`

```
image.find_features(cascade[, threshold=0.5[, scale=1.5[, roi]]])
```

该方法在图像中搜索与 Haar Cascade 模型匹配的所有区域，并返回这些特征的边界框矩形元组 `(x, y, w, h)` 的列表。若未找到任何特征，则返回空列表。

- `cascade` 是一个 Haar Cascade 对象，详细信息请参阅 `image.HaarCascade()`。
- `threshold` 是一个浮点数（范围为 0.0 至 1.0），较小的值可提高检测速率但可能增加误报率；相反，较高的值会降低检测速率同时降低误报率。
- `scale` 是一个大于 1.0 的浮点数。较高的比例因子运行速度较快，但图像匹配效果相对较差。理想值介于 1.35 和 1.5 之间。
- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持灰度图像。

1.100 `find_eye`

```
image.find_eye(roi)
```

该函数在指定的感兴趣区域 `(x, y, w, h)` 内查找瞳孔，并返回图像中瞳孔的位置元组 `(x, y)`。若未找到瞳孔，则返回 `(0, 0)`。

使用此函数前，需先通过 `image.find_features()` 和 Haar 算子 `frontalface` 搜索人脸，然后使用 `image.find_features()` 和 Haar 算子 `find_eye` 在人脸上搜索眼睛。最后，在调用 `image.find_features()` 返回的每个眼睛 ROI 上调用此方法，以获取瞳孔的坐标。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持灰度图像。

1.101 `find_lbp`

```
image.find_lbp(roi)
```

此函数从指定的 ROI 元组 `(x, y, w, h)` 中提取局部二值模式 (LBP) 关键点。您可以使用 `image.match_descriptor` 函数比较两组关键点，以获得匹配距离。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持灰度图像。

1.102 `find_keypoints`

```
image.find_keypoints([roi[, threshold=20[, normalized=False[, scale_factor=1.5[,
max_keypoints=100[, corner_detector=image.CORNER_AGAST]]]]]])
```

该函数从指定的 ROI 元组 `(x, y, w, h)` 中提取 ORB 关键点。您可以使用 `image.match_descriptor` 函数比较两组关键点以获取匹配区域。若未发现关键点，则返回 `None`。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。
- `threshold` 控制提取关键点的数量（取值范围为 0-255）。对于默认的 AGAST 角点检测器，该值应设为约 20；对于 FAST 角点检测器，该值应设为约 60 至 80。阈值越低，提取的角点越多。
- `normalized` 是布尔值。若为 `True`，则在多分辨率下关闭关键点提取。若您不关心处理扩展问题，且希望算法运行更快，则将其设置为 `True`。
- `scale_factor` 是一个大于 1.0 的浮点数。较高的比例因子运行速度较快，但图像匹配效果相对较差。理想值介于 1.35 和 1.5 之间。
- `max_keypoints` 是关键点对象能够容纳的最大关键点数量。若关键点对象过大导致内存问题，请适当降低该值。
- `corner_detector` 是提取关键点所使用的角点检测器算法。可选值为 `image.CORNER_FAST` 或 `image.CORNER_AGAST`。FAST 角点检测器速度较快，但准确度较低。

注意： 此方法仅支持灰度图像。

1.103 `find_edges`

```
image.find_edges(edge_type[, threshold])
```

该函数将图像转换为黑白图像，仅保留边缘为白色像素。

- `edge_type` 可选值包括：
  - `image.EDGE_SIMPLE` - 简单的阈值高通滤波算法
  - `image.EDGE_CANNY` - Canny 边缘检测算法
- `threshold` 是包含低阈值和高阈值的二元元组。您可以通过调整该值来控制边缘质量，默认设置为 `(100, 200)`。

注意： 此方法仅支持灰度图像。

1.104 `find_hog`

```
image.find_hog([roi[, size=8]])
```

此函数使用 HOG（定向梯度直方图）算法替换 ROI 中的像素。

- `roi` 是感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，默认 ROI 为整个图像。操作范围仅限于该区域内的像素。

注意： 此方法仅支持灰度图像。

1.105 `draw_ellipse`

```
image.draw_ellipse(cx, cy, rx, ry, color, thickness=1)
```

`draw_ellipse` 函数用于在图像上绘制一个椭圆。

- `cx, cy`：椭圆中心的坐标。
- `rx, ry`：椭圆在 x 轴和 y 轴

方向的半径。

- `color`：椭圆的颜色。
- `thickness`：椭圆边框的厚度（默认为 1）。

该函数返回图像对象，以便您可以使用 `.` 表示法调用其他方法。

注意： 此方法不支持压缩图像和 Bayer 图像。

OpenMV **原生** API 移植自 `openmv`，功能保持一致。用户可以参考原生文档获取更多 API 细节。

2. image **模块函数**

2.1 `rgb_to_lab`

将 RGB888 转换为 LAB 颜色空间。

```
image.rgb_to_lab(rgb_tuple)
```

2.2 `lab_to_rgb`

将 LAB 颜色空间转换为 RGB888。

```
image.lab_to_rgb(lab_tuple)
```

2.3 `rgb_to_grayscale`

将 RGB888 转换为灰度值。

```
image.rgb_to_grayscale(rgb_tuple)
```

2.4 `grayscale_to_rgb`

将灰度值转换为 RGB888。

```
image.grayscale_to_rgb(g_value)
```

2.5 `load_descriptor`

从文件加载描述符对象。

```
image.load_descriptor(path)
```

2.6 `save_descriptor`

保存描述符对象到文件。

```
image.save_descriptor(path, descriptor)
```

2.7 `match_descriptor`

比较两个描述符对象，返回匹配结果。

```
image.match_descriptor(descriptor0, descriptor1, threshold=70,
filter_outliers=False)
```

3 类 `ImageWriter`

`ImageWriter` 对象用于快速将未压缩的图像写入磁盘。

3.1 构造函数

```
class image.ImageWriter(path)
```

此构造函数创建一个 `ImageWriter` 对象，使您能够以 OpenMV Cam 所支持的简单文件格式将未压缩的图像写入磁盘。所写入的未压缩图像可通过 `ImageReader` 重新读取。

3.2 `size`

```
imagewriter.size()
```

此函数返回当前正在写入的文件的大小。

3.3 `add_frame`

```
imagewriter.add_frame(img)
```

该函数将一幅图像写入磁盘。由于图像未经过压缩，写入速度较快，但会占用较大的磁盘空间。

3.4 `close`

```
imagewriter.close()
```

此函数用于关闭图像流文件。请务必在操作完成后关闭文件，以避免文件损坏。

4 类 `ImageReader`

`ImageReader` 对象使您能够快速从磁盘中读取未压缩的图像。

4.1 构造函数

```
class image.ImageReader(path)
```

此构造函数创建一个 `ImageReader` 对象，用于回放由 `ImageWriter` 对象写入的图像数据。通过 `ImageWriter` 回放的帧将以与写入时相同的帧率 (FPS) 进行回放。

4.2 `size`

```
imagereader.size()
```

此函数返回当前正在读取的文件的大小。

4.3 `next_frame`

```
imagereader.next_frame([copy_to_fb=True, loop=True])
```

此函数从由 `ImageWriter` 写入的文件中返回图像对象。如果 `copy_to_fb` 为 `True`，则图像对象将直接加载到帧缓冲区中；否则，图像对象将存储在堆中。请注意，除非图像较小，否则堆可能没有足够的空间存储图像对象。如果 `loop` 为 `True`，则在读取完流中的最后一帧后，回放将重新开始；否则，在所有帧读取完毕后，此方法将返回 `None`。

注意：`imagereader.next_frame` 方法会尝试在读取帧后通过暂停播放来限制回放速度，以匹配帧的记录速度；否则，该方法可能以超过 200 FPS 的速度快速播放所有图像。

4.4 `close`

```
imagereader.close()
```

此函数用于关闭正在读取的文件。虽然文件是只读的，因此在未关闭的情况下不会损坏，但您仍然需要执行此操作，以确保 `imagereader` 对象的完整性。

5 类 `HaarCascade`

`HaarCascade` 特征描述符用于 `image.find_features()` 方法，不提供直接调用的方法。

5.1 构造函数

```
class image.HaarCascade(path[, stages=Auto])
```

此构造函数从 Haar Cascade 二进制文件（适合 OpenMV Cam 的格式）加载 Haar Cascade。如果您传入字符串 "frontalface" 而非路径，则构造函数将加载内置的正脸 Haar Cascade。同样，您可以通过传入 "eye" 来加载相应的 Haar Cascade。该方法返回加载的 Haar Cascade 对象，以供后续使用 `image.find_features()`。

`stages` 的默认值为 Haar Cascade 中的阶段数，但您可以指定较低的数值以加速特征检测器的运行，尽管这可能会增加误报率。

> 您可以制作适用于 OpenMV Cam 的自定义 Haar Cascades。首先，通过谷歌搜索 "Haar Cascade" 来查找是否已经有人为您想要检测的对象制作了 OpenCV Haar Cascade。如果没有，您可能需要自己动手制作（工作量较大）。关于如何制作自定义 Haar Cascade，请参见相关资料；关于如何将 OpenCV Haar Cascades 转化为 OpenMV Cam 可读取的格式，请参考相应脚本。

问： Haar Cascade **是什么?**

答： Haar Cascade 是一系列用于判断对象是否存在于图像中的对比检查。这些对比检查被分为多个阶段，后一阶段的执行依赖于前一阶段的结果。虽然对比检查本身并不复杂，例如检查图像中心的亮度是否低于边缘的亮度，但它们是高效的特征检测工具。初始阶段执行大范围检查，后续阶段则关注更小的区域。

问： Haar Cascades **是如何制作的?**

答： Haar Cascades 是通过带有正负标记的图像对生成算法进行训练而成。例如，使用数百张包含猫（标记为正例）的图像和数百张不含猫（标记为负例）的图像来训练该算法。最终生成的模型便是用于检测猫的 Haar Cascade。

6 类 `Similarity`

相似度对象由 `image.get_similarity` 函数返回。

6.1 构造函数

```
class image.similarity
```

请通过调用 `image.get_similarity()` 函数来创建此对象。

6.2 `mean`

```
similarity.mean()
```

此函数返回 8x8 像素块结构相似性差异的均值，范围为 [-1, +1]，其中 -1 表示完全不同，+1 表示完全相同。您也可以通过索引 [0] 直接获取该值。

6.3 `stdev`

```
similarity.stdev()
```

此函数返回 8x8 像素块结构相似性差异的标准偏差。您也可以通过索引 [1] 获取该值。

6.4 `min`

```
similarity.min()
```

此函数返回 8x8 像素块结构相似性差异的最小值，范围为 [-1, +1]，其中 -1 表示完全不同，+1 表示完全相同。您也可以通过索引 [2] 获取该值。

> 通过查看此值，您可以快速判断两个图像之间的任何 8x8 像素块是否存在显著差异，即值远低于 +1。

6.5 `max`

```
similarity.max()
```

此函数返回 8x8 像素块结构相似性差异的最大值，范围为 [-1, +1]，其中 -1 表示完全不同，+1 表示完全相同。您也可以通过索引 [3] 获取该值。

> 通过查看此值，您可以快速判断两个图像之间的任何 8x8 像素块是否完全相同，即值远大于 -1。

7 类 `Histogram`

直方图对象由 `image.get_histogram` 方法返回。灰度直方图包含多个标准化的二进制通道，其总和为 1。RGB565格式的直方图则有三个二进制通道，同样经过标准化处理，确保其总和为1。

7.1 构造函数

```
class image.histogram
```

请通过调用 `image.get_histogram()` 函数来创建该对象。

7.2 `bins`

```
histogram.bins()
```

返回灰度直方图的浮点数列表。您也可以通过索引 [0] 访问该值。

7.3 `l_bins`

```
histogram.l_bins()
```

返回RGB565格式直方图中LAB的L通道的浮点数列表。您可以通过索引 [0] 获取该值。

7.4 `a_bins`

```
histogram.a_bins()
```

返回RGB565格式直方图中LAB的A通道的浮点数列表。您可以通过索引 [1] 获取该值。

7.5 `b_bins`

```
histogram.b_bins()
```

返回RGB565格式直方图中LAB的B通道的浮点数列表。您可以通过索引 [2] 获取该值。

7.6 `get_percentile`

```
histogram.get_percentile(percentile)
```

计算直方图通道的累计分布函数（CDF），并返回指定百分位数（0.0 - 1.0）对应的直方图值。

例如，如果传入0.1，该方法将指示在累加过程中，哪个二进制值使累加器超过0.1。在未出现异常效用干扰自适应色跟踪结果的情况下，此方法对确定颜色分布的最小值（0.1）和最大值（0.9）尤为有效。

7.7 `get_threshold`

```
histogram.get_threshold()
```

采用Otsu方法计算最佳阈值，将直方图的每个通道一分为二。该方法返回一个 `image.threshold` 对象，特别适用于确定最佳的 `image.binary()` 阈值。

7.8 `get_statistics`

```
histogram.get_statistics()
```

计算直方图中每个颜色通道的平均值、中位数、众数、标准差、最小值、最大值、下四分位数和上四分位数，并返回一个 `statistics` 对象。您也可以使用 `histogram.statistics()` 和 `histogram.get_stats()` 作为该方法的别名。

8 类 `Percentile`

百分位值对象由 `histogram.get_percentile` 方法返回。灰度百分位值包含一个通道，不使用 `l_*`、`a_*` 或 `b_*` 方法。RGB565格式的百分位值包含三个通道，需使用 `l_*`、`a_*` 和 `b_*` 方法。

8.1 构造函数

```
class image.percentile
```

请通过调用 `histogram.get_percentile()` 函数来创建该对象。

8.2 `value`

```
percentile.value()
```

返回灰度百分位值（取值范围为0-255）。

您也可以通过索引 [0] 访问该值。

8.3 `l_value`

```
percentile.l_value()
```

返回RGB565格式中LAB的L通道的百分位值（取值范围为0-100）。

您也可以通过索引 [0] 获取该值。

8.4 `a_value`

```
percentile.a_value()
```

返回RGB565格式中LAB的A通道的百分位值（取值范围为-128至127）。

您也可以通过索引 [1] 获取该值。

8.5 `b_value`

```
percentile.b_value()
```

返回RGB565格式中LAB的B通道的百分位值（取值范围为-128至127）。

您也可以通过索引 [2] 获取该值。

9 类 `Threshold`

阈值对象由 `histogram.get_threshold` 方法返回。

灰度图像包含一个通道，不包含 `l_*`、`a_*` 和 `b_*` 方法。

RGB565格式的阈值包含三个通道，需使用 `l_*`、`a_*` 和 `b_*` 方法。

9.1 构造函数

```
class image.threshold
```

请通过调用 `histogram.get_threshold()` 函数来创建该对象。

9.2 `value`

```
threshold.value()
```

返回灰度图的阈值（范围在0至255之间）。

您也可以通过索引 [0] 获取该值。

9.3 `l_value`

```
threshold.l_value()
```

返回RGB565格式中LAB的L通道的阈值（范围在0至100之间）。

您也可以通过索引 [0] 获取该值。

9.4 `a_value`

```
threshold.a_value()
```

返回RGB565格式中LAB的A通道的阈值（范围在-128至127之间）。

您也可以通过索引 [1] 获取该值。

9.5 `b_value`

```
threshold.b_value()
```

返回RGB565格式中LAB的B通道的阈值（范围在-128至127之间）。

您也可以通过索引 [2] 获取该值。

10 类 `Statistics`

统计数据对象由 `histogram.get_statistics` 或 `image.get_statistics` 方法返回。

灰度统计数据包含一个通道，且不使用 `l_*`、`a_*` 或 `b_*` 方法。

RGB565格式的统计数据包含三个通道，需使用 `l_*`、`a_*` 和 `b_*` 方法。

10.1 构造函数

```
class image.statistics
```

请通过调用 `histogram.get_statistics()` 或 `image.get_statistics()` 函数来创建该对象。

10.2 `mean`

```
statistics.mean()
```

返回灰度均值（范围为0-255，类型为int）。

您也可以通过索引 [0] 获取该值。

10.3 `median`

```
statistics.median()
```

返回灰度中位数（范围为0-255，类型为int）。

您也可以通过索引 [1] 获取该值。

10.4 `mode`

```
statistics.mode()
```

返回灰度众数（范围为0-255，类型为int）。

您也可以通过索引 [2] 获取该值。

10.5 `stdev`

```
statistics.stdev()
```

返回灰度标准差（范围为0-255，类型为int）。

您也可以通过索引 [3] 获取该值。

10.6 `min`

```
statistics.min()
```

返回灰度最小值（范围为0-255，类型为int）。

您也可以通过索引 [4] 获取该值。

10.7 `max`

```
statistics.max()
```

返回灰度最大值（范围为0-255，类型为int）。

您也可以通过索引 [5] 获取该值。

10.8 `lq`

```
statistics.lq()
```

返回灰度下四分位数（范围为0-255，类型为int）。

您也可以通过索引 [6] 获取该值。

10.9 `uq`

```
statistics.uq()
```

返回灰度上四分位数（范围为0-255，类型为int）。

您也可以通过索引 [7] 获取该值。

10.10 `l_mean`

```
statistics.l_mean()
```

返回RGB565格式中LAB的L通道的均值（范围为0-255，类型为int）。

您也可以通过索引 [0] 获取该值。

10.11 `l_median`

```
statistics.l_median()
```

返回RGB565格式中LAB的L通道的中位数（范围为0-255，类型为int）。

您也可以通过索引 [1] 获取该值。

10.12 `l_mode`

```
statistics.l_mode()
```

返回RGB565格式中LAB的L通道的众数（范围为0-255，类型为int）。

您也可以通过索引 [2] 获取该值。

10.13 `l_stdev`

```
statistics.l_stdev()
```

返回RGB565格式中LAB的L通道

的标准差（范围为0-255，类型为int）。

您也可以通过索引 [3] 获取该值。

10.14 `l_min`

```
statistics.l_min()
```

返回RGB565格式中LAB的L通道的最小值（范围为0-255，类型为int）。

您也可以通过索引 [4] 获取该值。

10.15 `l_max`

```
statistics.l_max()
```

返回RGB565格式中LAB的L通道的最大值（范围为0-255，类型为int）。

您也可以通过索引 [5] 获取该值。

10.16 `l_lq`

```
statistics.l_lq()
```

返回RGB565格式中LAB的L通道下四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [6] 获取该值。

10.17 `l_uq`

```
statistics.l_uq()
```

返回RGB565格式中LAB的L通道上四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [7] 获取该值。

10.18 `a_mean`

```
statistics.a_mean()
```

返回RGB565格式中LAB的A通道均值，取值范围为0至255（类型为int）。您也可以通过索引 [8] 获取该值。

10.19 `a_median`

```
statistics.a_median()
```

返回RGB565格式中LAB的A通道中位数，取值范围为0至255（类型为int）。您也可以通过索引 [9] 获取该值。

10.20 `a_mode`

```
statistics.a_mode()
```

返回RGB565格式中LAB的A通道众数，取值范围为0至255（类型为int）。您也可以通过索引 [10] 获取该值。

10.21 `a_stdev`

```
statistics.a_stdev()
```

返回RGB565格式中LAB的A通道标准偏差，取值范围为0至255（类型为int）。您也可以通过索引 [11] 获取该值。

10.22 `a_min`

```
statistics.a_min()
```

返回RGB565格式中LAB的A通道最小值，取值范围为0至255（类型为int）。您也可以通过索引 [12] 获取该值。

10.23 `a_max`

```
statistics.a_max()
```

返回RGB565格式中LAB的A通道最大值，取值范围为0至255（类型为int）。您也可以通过索引 [13] 获取该值。

10.24 `a_lq`

```
statistics.a_lq()
```

返回RGB565格式中LAB的A通道下四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [14] 获取该值。

10.25 `a_uq`

```
statistics.a_uq()
```

返回RGB565格式中LAB的A通道上四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [15] 获取该值。

10.26 `b_mean`

```
statistics.b_mean()
```

返回RGB565格式中LAB的B通道均值，取值范围为0至255（类型为int）。您也可以通过索引 [16] 获取该值。

10.27 `b_median`

```
statistics.b_median()
```

返回RGB565格式中LAB的B通道中位数，取值范围为0至255（类型为int）。您也可以通过索引 [17] 获取该值。

10.28 `b_mode`

```
statistics.b_mode()
```

返回RGB565格式中LAB的B通道众数，取值范围为0至255（类型为int）。您也可以通过索引 [18] 获取该值。

10.29 `b_stdev`

```
statistics.b_stdev()
```

返回RGB565格式中LAB的B通道标准偏差，取值范围为0至255（类型为int）。您也可以通过索引 [19] 获取该值。

10.30 `b_min`

```
statistics.b_min()
```

返回RGB565格式中LAB的B通道最小值，取值范围为0至255（类型为int）。您也可以通过索引 [20] 获取该值。

10.31 `b_max`

```
statistics.b_max()
```

返回RGB565格式中LAB的B通道最大值，取值范围为0至255（类型为int）。您也可以通过索引 [21] 获取该值。

10.32 `b_lq`

```
statistics.b_lq()
```

返回RGB565格式中LAB的B通道下四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [22] 获取该值。

10.33 `b_uq`

```
statistics.b_uq()
```

返回RGB565格式中LAB的B通道上四分位数，取值范围为0至255（类型为int）。您也可以通过索引 [23] 获取该值。

11 类 `Blob`

色块对象由 `image.find_blobs` 方法返回。

11.1 构造函数

```
class image.blob
```

请通过调用 `image.find_blobs()` 函数以创建该对象。

11.2 `rect`

```
blob.rect()
```

返回一个矩形元组 (x, y, w, h)，用于色块边界框的图像绘制等其他图像处理方法，例如 `image.draw_rectangle`。

11.3 `x`

```
blob.x()
```

返回色块边界框的 x 坐标（类型为 int）。您也可以通过索引 [0] 获取该值。

11.4 `y`

```
blob.y()
```

返回色块边界框的 y 坐标（类型为 int）。您也可以通过索引 [1] 获取该值。

11.5 `w`

```
blob.w()
```

返回色块边界框的宽度（类型为 int）。您也可以通过索引 [2] 获取该值。

11.6 `h`

```
blob.h()
```

返回色块边界框的高度（类型为 int）。您也可以通过索引 [3] 获取该值。

11.6 `pixels`

```
blob.pixels()
```

返回属于该色块的像素数量（类型为 int）。您也可以通过索引 [4] 获取该值。

11.7 `cx`

```
blob.cx()
```

返回色块中心的 x 坐标（类型为 int）。您也可以通过索引 [5] 获取该值。

11.8 `cy`

```
blob.cy()
```

返回色块中心的 y 坐标（类型为 int）。您也可以通过索引 [6] 获取该值。

11.9 `rotation`

```
blob.rotation()
```

返回色块的旋转角度（单位：弧度）。对于类似铅笔或钢笔的色块，该值在0至180之间。如果色块是圆形的，则该值无效；如果色块具有完全的不对称性，则旋转角度范围为0至360度。您也可以通过索引 [7] 获取该值。

11.10 `code`

```
blob.code()
```

返回一个16位的二进制数，其中每个颜色阈值对应一位，表示该色块的属性。例如，如果通过 `image.find_blobs` 查找三个颜色阈值，则该色块可以设置为第0/1/2位。注意：除非在调用 `image.find_blobs` 时设置 `merge=True`，否则每个色块只能设置一位。因此，不同颜色阈值的多个色块可以合并在一起。您也可以利用该方法结合多个阈值实现颜色代码的跟踪。您也可以通过索引 [8] 获取该值。

11.11 `count`

```
blob.count()
```

返回合并为该色块的多个色块的数量。只有在调用 `image.find_blobs` 时设置 `merge=True`，此数字才会大于1。您也可以通过索引 [9] 获取该值。

11.12 `area`

```
blob.area()
```

返回色块周围边框的面积（计算方式为 w * h）。

11.13 `density`

```
blob.density()
```

返回色块的密度比，表示在色块边界框区域内的像素点数量。一般而言，较低的密度比表明该对象的锁定效果不佳。

12 类 `Line`

直线对象由 `image.find_lines`、`image.find_line_segments` 或 `image.get_regression` 方法返回。

12.1 构造函数

```
class image.line
```

请通过调用 `image.find_lines()`、`image.find_line_segments()` 或 `image.get_regression()` 函数以创建该对象。

12.2 `line`

```
line.line()
```

返回一个直线元组 (x1, y1, x2, y2)，用于图像绘制等其他图像处理方法，例如 `image.draw_line`。

12.3 `x1`

```
line.x1()
```

返回直线的第一个顶点 (p1) 的 x 坐标分量。您也可以通过索引 [0] 获取该值。

12.4 `y1`

```
line.y1()
```

返回直线的第一个顶点 (p1) 的 y 坐标分量。您也可以通过索引 [1] 获取该值。

12.5 `x2`

```
line.x2()
```

返回直线的第二个顶点 (p2) 的 x 坐标分量。您也可以通过索引 [2] 获取该值。

12.6 `y2`

```
line.y2()
```

返回直线的第二个顶点 (p2) 的 y 坐标分量。您也可以通过索引 [3] 获取该值。

12.7 `length`

```
line.length()
```

返回直线的长度，计算方式为 (\sqrt{((x2-x1)^2) + ((y2-y1)^2)})。您也可以通过索引 [4] 获取该值。

12.8 `magnitude`

```
line.magnitude()
```

返回霍夫变换后直线的长度。您也可以通过索引 [5] 获取该值。

12.9 `theta`

```
line.theta()
```

返回霍夫变换后直线的角度（范围：0-179度）。您也可以通过索引 [7] 获取该值。

12.10 `rho`

```
line.rho()
```

返回霍夫变换后直线的ρ值。您也可以通过索引 [8] 获取该值。

13 类 `Circle`

圆形对象由 `image.find_circles` 方法返回。

13.1 构造函数

```
class image.circle
```

请通过调用 `image.find_circles()` 函数以创建该对象。

13.2 `x`

```
circle.x()
```

返回圆心的 x 坐标。您也可以通过索引 [0] 获取该值。

13.3 `y`

```
circle.y()
```

返回圆心的 y 坐标。您也可以通过索引 [1] 获取该值。

13.4 `r`

```
circle.r()
```

返回圆的半径。您也可以通过索引 [2] 获取该值。

13.5 `magnitude`

```
circle.magnitude()
```

返回圆的大小。您也可以通过索引 [3] 获取该值。

14 类 `Rect`

矩形对象由 `image.find_rects` 函数返回。

14.1 构造函数

```
class image.rect
```

请使用 `image.find_rects()` 函数创建此对象。

14.2 `corners`

```
rect.corners()
```

该方法返回一个包含矩形对象四个角的元组列表，每个元组格式为 (x, y)。四个角的顺序通常为从左上角开始，按顺时针方向排列。

14.3 `rect`

```
rect.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的边界框。

14.4 `x`

```
rect.x()
```

该方法返回矩形左上角的 x 坐标。您也可以通过索引 [0] 获取该值。

14.5 `y`

```
rect.y()
```

该方法返回矩形左上角的 y 坐标。您也可以通过索引 [1] 获取该值。

14.6 `w`

```
rect.w()
```

该方法返回矩形的宽度。您也可以通过索引 [2] 获取该值。

14.7 `h`

```
rect.h()
```

该方法返回矩形的高度。您也可以通过索引 [3] 获取该值。

14.8 `magnitude`

```
rect.magnitude()
```

该方法返回矩形的大小。您也可以通过索引 [4] 获取该值。

15 类 `QRCode`

二维码对象由 `image.find_qrcodes` 函数返回。

15.1 构造函数

```
class image.qrcode
```

请使用 `image.find_qrcodes()` 函数创建此对象。

15.2 `corners`

```
qrcode.corners()
```

该方法返回一个包含二维码四个角的元组列表，每个元组格式为 (x, y)。四个角的顺序通常为从左上角开始，按顺时针方向排列。

15.3 `rect`

```
qrcode.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的二维码边界框。

15.4 `x`

```
qrcode.x()
```

该方法返回二维码边界框的 x 坐标 (int)。您也可以通过索引 [0] 获取该值。

15.5 `y`

```
qrcode.y()
```

该方法返回二维码边界框的 y 坐标 (int)。您也可以通过索引 [1] 获取该值。

15.6 `w`

```
qrcode.w()
```

该方法返回二维码边界框的宽度 (int)。您也可以通过索引 [2] 获取该值。

15.7 `h`

```
qrcode.h()
```

该方法返回二维码边界框的高度 (int)。您也可以通过索引 [3] 获取该值。

15.8 `payload`

```
qrcode.payload()
```

该方法返回二维码的有效载荷字符串，例如 URL。您也可以通过索引 [4] 获取该值。

15.9 `version`

```
qrcode.version()
```

该方法返回二维码的版本号 (int)。您也可以通过索引 [5] 获取该值。

15.10 `ecc_level`

```
qrcode.ecc_level()
```

该方法返回二维码的错误纠正水平 (int)。您也可以通过索引 [6] 获取该值。

15.11 `mask`

```
qrcode.mask()
```

该方法返回二维码的掩码 (int)。您也可以通过索引 [7] 获取该值。

15.12 `data_type`

```
qrcode.data_type()
```

该方法返回二维码的数据类型。您也可以通过索引 [8] 获取该值。

15.13 `eci`

```
qrcode.eci()
```

该方法返回二维码的 ECI（编码指示），用于存储 QR 码中数据字节的编码。处理包含非标准 ASCII 文本的二维码时，需查看此值。您也可以通过索引 [9] 获取该值。

15.14 `is_numeric`

```
qrcode.is_numeric()
```

若二维码的数据类型为数字格式，则返回 True。

15.15`is_alphanumeric`

```
qrcode.is_alphanumeric()
```

若二维码的数据类型为字母数字格式，则返回 True。

15.16 `is_binary`

```
qrcode.is_binary()
```

若二维码的数据类型为二进制格式，则返回 True。若需准确处理所有类型的文本，请检查 `eci` 是否为 True，以确定数据的文本编码。通常为标准 ASCII，但可能是包含两个字节字符的 UTF-8。

15.17 `is_kanji`

```
qrcode.is_kanji()
```

若二维码的数据类型为日文汉字格式，则返回 True。若返回值为 True，需自行解码字符串，因为日文汉字每个字符为 10 位，而 MicroPython 不支持解析此类文本。

16 类 `AprilTag`

AprilTag 对象由 `image.find_apriltags` 函数返回。

16.1 构造函数

```
class image.apriltag
```

请使用 `image.find_apriltags()` 函数创建此对象。

16.2 `corners`

```
apriltag.corners()
```

该方法返回一个包含 AprilTag 四个角的元组列表，每个元组格式为 (x, y)。四个角的顺序通常为从左上角开始，按顺时针方向排列。

16.3 `rect`

```
apriltag.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的 AprilTag 边界框。

16.4 `x`

```
apriltag.x()
```

该方法返回 AprilTag 边界框的 x 坐标 (int)。您也可以通过索引 [0] 获取该值。

16.5 `y`

```
apriltag.y()
```

该方法返回 AprilTag 边界框的 y 坐标 (int)。您也可以通过索引 [1] 获取该值。

16.6 `w`

```
apriltag.w()
```

该方法返回 AprilTag 边界框的宽度 (int)。您也可以通过索引 [2] 获取该值。

16.7 `h`

```
apriltag.h()
```

该方法返回 AprilTag 边界框的高度 (int)。您也可以通过索引 [3] 获取该值。

16.8 `id`

```
apriltag.id()
```

该方法返回 AprilTag 的数字 ID。

- TAG16H5 -> 0 到 29
- TAG25H7 -> 0 到 241
- TAG25H9 -> 0 到 34
- TAG36H10 -> 0 到 2319
- TAG36H11 -> 0 到 586
- ARTOOLKIT -> 0 到 511

您也可以通过索引 [4] 获取该值。

16.9 `family`

```
apriltag.family()
```

该方法返回 AprilTag 的数字家族。

- image.TAG16H5
- image.TAG25H7
- image.TAG25H9
- image.TAG36H10
- image.TAG36H11
- image.ARTOOLKIT

您也可以通过索引 [5] 获取该值。

16.10 `cx`

```
apriltag.cx()
```

该方法返回 AprilTag 的中心 x 坐标 (int)。您也可以通过索引 [6] 获取该值。

16.11 `cy`

```
apriltag.cy()
```

该方法返回 AprilTag 的中心 y 坐标 (int)。您也可以通过索引 [7] 获取该值。

16.12 `rotation`

```
apriltag.rotation()
```

该方法返回 AprilTag 的旋转角度，以弧度计量 (int)。您也可以通过索引 [8] 获取该值。

16.13 `decision_margin`

```
apriltag.decision_margin()
```

该方法返回 AprilTag 的决策边际，反映了对检测的置信度 (float)。您也可以通过索引 [9] 获取该值。

16.14 `hamming`

```
apriltag.hamming()
```

该方法返回 AprilTag 可接受的最大汉明距离（即可接受的数位误差）。具体如下：

- TAG16H5：最多可接受 0 位错误
- TAG25H7：最多可接受 1 位错误
- TAG25H9：最多可接受 3 位错误
- TAG36H10：最多可接受 3 位错误
- TAG36H11：最多可接受 4 位错误
- ARTOOLKIT：最多可接受 0 位错误

您可以通过索引 [10] 获取该值。

16.15 `goodness`

```
apriltag.goodness()
```

该方法返回 AprilTag 图像的色饱和度，取值范围为 0.0 到 1.0，其中 1.0 表示最佳状态。

> 目前该数值通常为 0.0。未来我们计划启用名为“标签细化”的功能，以支持对更小的 AprilTag 的检测。然而，目前此功能可能会导致帧速率降低至 1 FPS 以下。

您可以通过索引 [11] 获取该值。

16.16 `x_translation`

```
apriltag.x_translation()
```

该方法返回摄像机在 x 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 x 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从左至右。

您可以通过索引 [12] 获取该值。

16.17 `y_translation`

```
apriltag.y_translation()
```

该方法返回摄像机在 y 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 y 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从上至下。

您可以通过索引 [13] 获取该值。

16.18 `z_translation`

```
apriltag.z_translation()
```

该方法返回摄像机在 z 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 z 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从前至后。

您可以通过索引 [14] 获取该值。

16.19 `x_rotation`

```
apriltag.x_rotation()
```

该方法返回 AprilTag 在 x 平面上的旋转角度，以弧度计量。例如，若从左至右移动摄像头观察 AprilTag，则可应用此方法。

您可以通过索引 [15] 获取该值。

16.20 `y_rotation`

```
apriltag.y_rotation()
```

该方法返回 AprilTag 在 y 平面上的旋转角度，以弧度计量。例如，若从上至下移动摄像头观察 AprilTag，则可应用此方法。

您可以通过索引 [16] 获取该值。

16.21 `z_rotation`

```
apriltag.z_rotation()
```

该方法返回 AprilTag 在 z 平面上的旋转角度，以弧度计量。例如，若旋转摄像头观察 AprilTag，则可应用此方法。

注意：此方法为 `apriltag.rotation()` 的重命名版本。

您可以通过索引 [17] 获取该值。

17 类 `DataMatrix`

数据矩阵对象由 `image.find_datamatrices` 函数返回。

17.1 构造函数

```
class image.datamatrix
```

请调用 `image.find_datamatrices()` 函数来创建此对象。

17.2 `corners`

```
datamatrix.corners()
```

该方法返回一个包含数据矩阵四个角的元组列表，每个元组格式为 (x, y)。四个角通常按从左上角开始，沿顺时针方向排列。

17.3 `rect`

```
datamatrix.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的数据矩阵边界框。

17.4 `x`

```
datamatrix.x()
```

该方法返回数据矩阵边界框的 x 坐标（整数）。您也可以通过索引 [0] 获取该值。

17.5 `y`

```
datamatrix.y()
```

该方法返回数据矩阵边界框的 y 坐标（整数）。您也可以通过索引 [1] 获取该值。

17.6 `w`

```
datamatrix.w()
```

该方法返回数据矩阵边界框的宽度（整数）。您也可以通过索引 [2] 获取该值。

17.7 `h`

```
datamatrix.h()
```

该方法返回数据矩阵边界框的高度（整数）。您也可以通过索引 [3] 获取该值。

17.8 `payload`

```
datamatrix.payload()
```

该方法返回数据矩阵的有效载荷字符串。例如：“字符串”。

您也可以通过索引 [4] 获取该值。

17.9 `rotation`

```
datamatrix.rotation()
```

该方法返回数据矩阵的旋转角度（以弧度计量，浮点数）。

您也可以通过索引 [5] 获取该值。

17.10 `rows`

```
datamatrix.rows()
```

该方法返回数据矩阵的行数（整数）。

您也可以通过索引 [6] 获取该值。

17.11 `columns`

```
datamatrix.columns()
```

该方法返回数据矩阵的列数（整数）。

您也可以通过索引 [7] 获取该值。

17.12 `capacity`

```
datamatrix.capacity()
```

该方法返回数据矩阵能够容纳的字符数量。

您也可以通过索引 [8] 获取该值。

17.13 `padding`

```
datamatrix.padding()
```

该方法返回数据矩阵中未使用的字符数量。

您也可以通过索引 [9] 获取该值。

18 类 `BarCode`

条形码对象由 `image.find_barcodes` 函数返回。

18.1 构造函数

```
class image.barcode
```

请调用 `image.find_barcodes()` 函数来创建此对象。

18.2 `corners`

```
barcode.corners()
```

该方法返回一个包含条形码四个角的元组列表，每个元组格式为 (x, y)。四个角通常按从左上角开始，沿顺时针方向排列。

18.3 `rect`

```
barcode.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的条形码边界框。

18.4 `x`

```
barcode.x()
```

该方法返回条形码边界框的 x 坐标（整数）。您也可以通过索引 [0] 获取该值。

18.5 `y`

```
barcode.y()
```

该方法返回条形码边界框的 y 坐标（整数）。您也可以通过索引 [1] 获取该值。

18.6 `w`

```
barcode.w()
```

该方法返回条形码边界框的宽度（整数）。您也可以通过索引 [2] 获取该值。

18.7 `h`

```
barcode.h()
```

该方法返回条形码边界框的高度（整数）。您也可以通过索引 [3] 获取该值。

18.8 `payload`

```
barcode.payload()
```

该方法返回条形码的有效载荷字符串，例如：“数量”。

您也可以通过索引 [4] 获取该值。

18.9 `type`

```
barcode.type()
```

该方法返回条形码的类型（整数）。可能的类型包括：

- image.EAN2
- image

.EAN5

- image.EAN8
- image.UPCE
- image.ISBN10
- image.UPCA
- image.EAN13
- image.ISBN13
- image.I25
- image.DATABAR
- image.DATABAR_EXP
- image.CODABAR
- image.CODE39
- image.PDF417（未来启用，当前尚不可用）
- image.CODE93
- image.CODE128

您也可以通过索引 [5] 获取该值。

18.10 `rotation`

```
barcode.rotation()
```

该方法返回条形码的旋转角度（以弧度计量，浮点数）。

您也可以通过索引 [6] 获取该值。

18.11 `quality`

```
barcode.quality()
```

该方法返回条形码在图像中被检测到的次数（整数）。

在扫描条形码时，每一条新的扫描线都能解码相同的条形码。每次进行此过程时，条形码的值都会增加。

您也可以通过索引 [7] 获取该值。

19 类 `Displacement`

位移对象由 `image.find_displacement` 函数返回。

19.1 构造函数

```
class image.displacement
```

请通过调用 `image.find_displacement()` 函数来创建此对象。

19.2 `x_translation`

```
displacement.x_translation()
```

该方法返回两个图像之间的 x 方向平移量，以像素为单位。返回值为浮点数，表示精确的子像素位移。

您也可以通过索引 [0] 获取该值。

19.3 `y_translation`

```
displacement.y_translation()
```

该方法返回两个图像之间的 y 方向平移量，以像素为单位。返回值为浮点数，表示精确的子像素位移。

您也可以通过索引 [1] 获取该值。

19.4 `rotation`

```
displacement.rotation()
```

该方法返回两个图像之间的旋转量，以弧度为单位。返回值为浮点数，表示精确的子像素旋转。

您也可以通过索引 [2] 获取该值。

19.5 `scale`

```
displacement.scale()
```

该方法返回两个图像之间的缩放比例，以浮点数表示。

您也可以通过索引 [3] 获取该值。

19.6 `response`

```
displacement.response()
```

该方法返回两幅图像之间位移匹配结果的质量评估，值范围为 0 到 1。若返回值小于 0.1，则该位移对象可能被视为噪声。

您也可以通过索引 [4] 获取该值。

20 类 `Kptmatch`

特征点对象由 `image.match_descriptor` 函数返回。

20.1 构造函数

```
class image.kptmatch
```

请通过调用 `image.match_descriptor()` 函数来创建此对象。

20.2 `rect`

```
kptmatch.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于绘制特征点边界框的其他图像处理方法，如 `image.draw_rectangle`。

20.3 `cx`

```
kptmatch.cx()
```

该方法返回特征点中心的 x 坐标，类型为整数。

您也可以通过索引 [0] 获取该值。

20.4 `cy`

```
kptmatch.cy()
```

该方法返回特征点中心的 y 坐标，类型为整数。

您也可以通过索引 [1] 获取该值。

20.5 `x`

```
kptmatch.x()
```

该方法返回特征点边界框的 x 坐标，类型为整数。

您也可以通过索引 [2] 获取该值。

20.6 `y`

```
kptmatch.y()
```

该方法返回特征点边界框的 y 坐标，类型为整数。

您也可以通过索引 [3] 获取该值。

20.7 `w`

```
kptmatch.w()
```

该方法返回特征点边界框的宽度，类型为整数。

您也可以通过索引 [4] 获取该值。

20.8 `h`

```
kptmatch.h()
```

该方法返回特征点边界框的高度，类型为整数。

您也可以通过索引 [5] 获取该值。

20.9 `count`

```
kptmatch.count()
```

该方法返回匹配特征点的数量，类型为整数。

您也可以通过索引 [6] 获取该值。

20.10 `theta`

```
kptmatch.theta()
```

该方法返回估计的特征点的旋度，类型为整数。

您也可以通过索引 [7] 获取该值。

20.11 `match`

```
kptmatch.match()
```

该方法返回匹配关键点的 (x, y) 元组列表。

您也可以通过索引 [8] 获取该值。

21 常量

21.1 `image.SEARCH_EX`

用于执行详尽的模板匹配搜索。

21.2 `image.SEARCH_DS`

用于执行更快的模板匹配搜索。

21.3 `image.EDGE_CANNY`

应用 Canny 边缘检测算法对图像进行边缘检测。

21.4 `image.EDGE_SIMPLE`

使用阈值高通滤波算法对图像进行边缘检测。

21.5 `image.CORNER_FAST`

用于 ORB 特征点的高速低准确率角点检测算法。

21.6 `image.CORNER_AGAST`

用于 ORB 特征点的低速高准确率角点检测算法。

21.7 `image.TAG16H5`

用于 AprilTags 的 TAG16H5 标签组位掩码枚举。

21.8 `image.TAG25H7`

用于 AprilTags 的 TAG25H7 标签组位掩码枚举。

21.9 `image.TAG25H9`

用于 AprilTags 的 TAG25H9 标签组位掩码枚举。

21.10 `image.TAG36H10`

用于 AprilTags 的 TAG36H10 标签组位掩码枚举。

21.11 `image.TAG36H11`

用于 AprilTags 的 TAG36H11 标签组位掩码枚举。

21.12 `image.ARTOOLKIT`

用于 AprilTags 的 ARTOLKIT 标签组位掩码枚举。

21.13 `image.EAN2`

EAN2 条形码类型的枚举。

21.14 `image.EAN5`

EAN5 条形码类型的枚举。

21.15 `image.EAN8`

EAN8 条形码类型的枚举。

21.16 `image.UPCE`

UPCE 条形码类型的枚举。

21.17 `image.ISBN10`

ISBN10 条形码类型的枚举。

21.18 `image.UPCA`

UPCA 条形码类型的枚举。

21.19 `image.EAN13`

EAN13 条形码类型的枚举。

21.20 `image.ISBN13`

ISBN13 条形码类型的枚举。

21.21 `image.I25`

I25 条形码类型的枚举。

21.22 `image.DATABAR`

DATABAR 条形码类型的枚举。

21.23 `image.DATABAR_EXP`

DATABAR_EXP 条形码类型的枚举。

21.24 `image.CODABAR`

CODABAR 条形码类型的枚举。

21.25 `image.CODE39`

CODE39 条形码类型的枚举。

21.26 `image.PDF417`

PDF417 条形码类型的枚举（目前尚未实现）。

21.27 `image.CODE93`

CODE93 条形码类型的枚举。

21.28 `image.CODE128`

CODE128 条形码类型的枚举。